**Практическое занятие №5**

# Модель ролевого управления доступом RBAC

**Проектирование и реализация ролевой модели управления доступом RBAC в базе данных.**

## Краткая теория

**RBAC (Role-Based Access Control)** — модель управления доступом, при которой права назначаются не непосредственно пользователям, а ролям.

Общая схема:

```
Пользователь
      ↓
    Роль
      ↓
   Права
      ↓
   Ресурсы БД
```

Например:

```
teacher01
     ↓
teacher_role
     ↓
SELECT students
SELECT grades
UPDATE grades
```

Вместо того чтобы отдельно выдавать права каждому преподавателю, администратор создаёт одну роль `teacher_role`, а затем назначает её всем преподавателям.

## Исходная ситуация

Представим информационную систему университета.

В базе данных существуют следующие таблицы:

```
students
teachers
groups
grades
payments
reports
users
```

В системе работают:

| Пользовательская категория | Количество |
|---|---|
| Студенты | 500 |
| Преподаватели | 50 |
| Бухгалтеры | 10 |
| Сотрудники деканата | 5 |
| Аналитики | 3 |
| Администраторы БД | 2 |

Вместо индивидуального назначения прав необходимо использовать **RBAC**.

Если база данных ещё не создана, создайте учебные таблицы:

```
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    full_name VARCHAR(100),
    group_name VARCHAR(50),
    email VARCHAR(100)
);

CREATE TABLE teachers (
    id SERIAL PRIMARY KEY,
    full_name VARCHAR(100),
    department VARCHAR(100)
);

CREATE TABLE groups (
    id SERIAL PRIMARY KEY,
    group_name VARCHAR(50),
    specialty VARCHAR(100)
);

CREATE TABLE grades (
    id SERIAL PRIMARY KEY,
    student_id INT,
    teacher_id INT,
    subject VARCHAR(100),
    grade INT
);

CREATE TABLE payments (
    id SERIAL PRIMARY KEY,
    student_id INT,
    amount DECIMAL(10,2),
    payment_date DATE
);

CREATE TABLE reports (
    id SERIAL PRIMARY KEY,
    report_name VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(100),
    role_name VARCHAR(50)
```

## Задание 1. Проектирование RBAC-модели

Определите роли, которые необходимы университетской системе.

Минимально должны присутствовать:

1. `student_role`
2. `teacher_role`
3. `accountant_role`
4. `dean_role`
5. `analyst_role`
6. `admin_role`

Заполните таблицу:

| Роль | Назначение | Необходимые ресурсы |
|---|---|---|
| Student | Работа со своими учебными данными | students, grades |
| Teacher | Работа с учебным процессом | students, teachers, groups, grades |
| Accountant | Работа с оплатами | payments |
| Dean | Работа с учебной информацией | students, teachers, groups, grades, reports |
| Analyst | Анализ данных | students, grades, payments, reports |
| Administrator | Администрирование БД | Все объекты |

## Задание 2. Создание ролей

Создайте роли PostgreSQL:

```
CREATE ROLE student_role;
CREATE ROLE teacher_role;
CREATE ROLE accountant_role;
CREATE ROLE dean_role;
CREATE ROLE analyst_role;
CREATE ROLE admin_role;
```

Проверьте созданные роли:

```
\du
```

или:

```
SELECT rolname
FROM pg_roles;
```

## Дополнительное задание

Создайте отдельных пользователей:

```
CREATE USER student01
WITH PASSWORD 'Student_Strong_123!';

CREATE USER teacher01
WITH PASSWORD 'Teacher_Strong_123!';

CREATE USER accountant01
WITH PASSWORD 'Accountant_Strong_123!';

CREATE USER dean01
WITH PASSWORD 'Dean_Strong_123!';

CREATE USER analyst01
WITH PASSWORD 'Analyst_Strong_123!';
```

## Задание 3. Назначение прав ролям

Теперь права предоставляются **ролям**, а не непосредственно пользователям.

## 3.1. Роль студента

Студенту необходим просмотр учебной информации.

```
GRANT SELECT ON students TO student_role;
GRANT SELECT ON grades TO student_role;
```

Студент **не должен** иметь возможность изменять оценки.

## 3.2. Роль преподавателя

Преподавателю необходим доступ к учебным данным:

```
GRANT SELECT ON students TO teacher_role;

GRANT SELECT ON teachers TO teacher_role;

GRANT SELECT ON groups TO teacher_role;

GRANT SELECT, INSERT, UPDATE ON grades TO teacher_role;
```

При этом преподаватель не должен удалять оценки.

## 3.3. Роль бухгалтера

Бухгалтер работает только с финансовой информацией:

```
GRANT SELECT, INSERT, UPDATE ON payments
TO accountant_role;
```

Бухгалтер не должен иметь доступ к изменению оценок.

## 3.4. Роль деканата

Деканат работает с учебной информацией:

```
GRANT SELECT ON students TO dean_role;

GRANT SELECT ON teachers TO dean_role;

GRANT SELECT ON groups TO dean_role;

GRANT SELECT, INSERT, UPDATE ON grades
TO dean_role;

GRANT SELECT, INSERT, UPDATE ON reports
TO dean_role;
```

## 3.5. Роль аналитика

Аналитику необходим просмотр информации для формирования аналитических отчётов:

```
GRANT SELECT ON students TO analyst_role;

GRANT SELECT ON grades TO analyst_role;

GRANT SELECT ON payments TO analyst_role;

GRANT SELECT ON reports TO analyst_role;
```

Аналитик **не должен** изменять данные.

# Задание 4. Назначение ролей пользователям

Теперь необходимо связать пользователей с ролями.

```
GRANT student_role TO student01;

GRANT teacher_role TO teacher01;

GRANT accountant_role TO accountant01;

GRANT dean_role TO dean01;

GRANT analyst_role TO analyst01;
```

Получаем:

```
student01
     ↓
student_role
     ↓
SELECT students
SELECT grades
teacher01
```

```
     ↓
teacher_role
     ↓
SELECT students
SELECT teachers
SELECT groups
SELECT/INSERT/UPDATE grades
```

## Задание 5. Иерархия ролей

RBAC может использовать иерархию ролей.

Создайте базовую роль:

```
CREATE ROLE employee_role;
```

Предоставьте ей общий доступ к некоторым данным:

```
GRANT SELECT ON students TO employee_role;
```

Теперь создайте роль:

```
CREATE ROLE senior_employee_role;
```

Назначьте базовую роль:

```
GRANT employee_role TO senior_employee_role;
```

Добавьте дополнительные права:

```
GRANT SELECT ON teachers TO senior_employee_role;
GRANT SELECT ON groups TO senior_employee_role;
```

Получается:

```
        senior_employee_role
                 ↓
          employee_role
                 ↓
          SELECT students
```

А дополнительно:

```
senior_employee_role
        ↓
SELECT teachers
SELECT groups
```

## Задание 6. Разделение обязанностей

Рассмотрим ситуацию.

Один сотрудник одновременно имеет:

```
accountant_role
```

```
+
admin_role
```

Это создаёт риск, поскольку один человек может:

- изменить финансовые данные;
- изменить права доступа;
- удалить данные;
- скрыть следы своих действий.

### Задание

Определите, какие роли **нежелательно совмещать**.

Заполните таблицу:

| Роль 1 | Роль 2 | Можно ли совмещать? | Почему? |
|---|---|---|---|
| Accountant | Administrator | ? | ? |
| Teacher | Student | ? | ? |
| Analyst | Accountant | ? | ? |
| Dean | Teacher | ? | ? |

## Задание 7. Проверка прав

Необходимо проверить, действительно ли RBAC работает.

Например, пользователь:

```
student01
```

имеет:

```
student_role
```

Попробуйте выполнить:

```
SELECT *
FROM students;
```

Операция должна быть разрешена.

Теперь:

```
UPDATE students
SET full_name = 'Test Student'
WHERE id = 1;
```

Операция должна быть запрещена, если студенту не выданы права `UPDATE`.

## Проверка преподавателя

Пользователь:

```
teacher01
```

должен иметь возможность:

```
SELECT *
FROM students;
```

и:

```
UPDATE grades
SET grade = 90
WHERE id = 1;
```

Но не должен иметь возможность:

```
DELETE FROM grades
WHERE id = 1;
```

если право `DELETE` ему не предоставлено.

## Проверка аналитика

Пользователь:

```
analyst01
```

может:

```
SELECT *
FROM payments;
```

Но не должен иметь возможность:

```
UPDATE payments
SET amount = 0
WHERE id = 1;
```

## Итоговая матрица RBAC

После выполнения заданий сформируйте итоговую таблицу:

| Роль | Students | Teachers | Groups | Grades | Payments | Reports |
|---|---|---|---|---|---|---|
| Student | R | — | — | R | — | — |
| Teacher | R | R | R | R/I/U | — | — |
| Accountant | — | — | — | — | R/I/U | — |

| | | | | | | |
|---|---|---|---|---|---|---|
| Dean | R | R | R | R/I/U | — | R/I/U |
| Analyst | R | — | — | R | R | R |
| Administrator | Полный доступ | Полный доступ | Полный доступ | Полный доступ | Полный доступ | Полный доступ |

Где:

- **R** — Read / SELECT;
- **I** — Insert / INSERT;
- **U** — Update / UPDATE;
- **D** — Delete / DELETE.

## Что должно быть в отчёте

Отчёт должен содержать:

## 1. Цель работы

Кратко описать цель практического занятия.

## 2. Исходную модель RBAC

Показать:

```
Пользователь → Роль → Права → Ресурс
```

## 3. Созданные роли

Например:

```
student_role
teacher_role
accountant_role
dean_role
analyst_role
admin_role
```

## 4. SQL-команды

Привести основные команды:

```
CREATE ROLE
CREATE USER
GRANT
REVOKE
```

## 5. Матрицу доступа

Предоставить итоговую таблицу ролей и разрешений.

## 6. Результаты проверки

Показать минимум:

- 3 разрешённые операции;
- 3 запрещённые операции.

## 7. Иерархию ролей

Показать пример:

```
senior_employee_role
        ↓
employee_role
```

## 8. Разделение обязанностей

Указать минимум одну пару ролей, совмещение которых создаёт повышенный риск.

## 9. Вывод

Ответить на вопрос:

**Как RBAC упрощает управление безопасностью базы данных?**